SKILL-13

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
   int fd;
   int saved_stdin;
   char line[256];

   /* Save the original standard input */
   saved_stdin = dup(STDIN_FILENO);

   if (saved_stdin == -1)
   {
      perror("dup stdin");
      return 1;
   }

   /* Open the input file */
   fd = open("input.txt", O_RDONLY);

   if (fd == -1)
   {
      perror("open input.txt");
      close(saved_stdin);
```

```
    return 1;
}

/* Redirect stdin to the file */
if (dup2(fd, STDIN_FILENO) == -1)
{
    perror("dup2 stdin");
    close(fd);
    close(saved_stdin);
    return 1;
}

close(fd);

printf("Reading from input.txt:\n");

while (fgets(line, sizeof(line), stdin) != NULL)
{
    printf("Read: %s", line);
}

/* Restore the original stdin */
if (dup2(saved_stdin, STDIN_FILENO) == -1)
{
    perror("restore stdin");
    close(saved_stdin);
    return 1;
}
```

```
    close(saved_stdin);

    printf("\nInput stream restored.\n");

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ printf "Operating Systems\nLinux System
Calls\nInput Redirection\n" > input.txt
cat input.txt
Operating Systems
Linux System Calls
Input Redirection
snigdha@Snigdha:~/quotes_practical$ nano input_redirect.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -o input_redirect inpu
t_redirect.c
./input_redirect
Reading from input.txt:
Read: Operating Systems
Read: Linux System Calls
Read: Input Redirection

Input stream restored.
snigdha@Snigdha:~/quotes_practical$ mv input.txt input_backup.txt
./input_redirect
open input.txt: No such file or directory
snigdha@Snigdha:~/quotes_practical$ mv input_backup.txt input.txt
snigdha@Snigdha:~/quotes_practical$
```

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>


int main(void)

{

    int fd;
```

```
int saved_stdout;

/* Save the original standard output */
saved_stdout = dup(STDOUT_FILENO);

if (saved_stdout == -1)
{
   perror("dup stdout");
   return 1;
}

/* Create file or truncate an existing file */
fd = open("output.txt",
        O_WRONLY | O_CREAT | O_TRUNC,
        0644);

if (fd == -1)
{
   perror("open output.txt");
   close(saved_stdout);
   return 1;
}

/* Redirect stdout to the file */
if (dup2(fd, STDOUT_FILENO) == -1)
{
   perror("dup2 stdout");
   close(fd);
```

```
    close(saved_stdout);
    return 1;
}

close(fd);

printf("Operating Systems Practical\n");
printf("Output redirection using dup2()\n");
printf("File output completed successfully.\n");

/* Ensure buffered output reaches the file */
if (fflush(stdout) == EOF)
{
    perror("fflush");
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    return 1;
}

/* Restore original stdout */
if (dup2(saved_stdout, STDOUT_FILENO) == -1)
{
    perror("restore stdout");
    close(saved_stdout);
    return 1;
}

close(saved_stdout);
```

```
    printf("Output stream restored.\n");

    return 0;
}
```

